

12 May 26'

DB CP & Assigned

9:08pm Tues

(1)

Select name from Student Where student_id
IN (Select student_id from Enrollments
Where course_id = (Select course_id
from Courses Where course_name = 'Database
Systems'));

(2)

Select name from Student Where
marks > ANY (Select marks from Student
Where section = 'B');

(3)

Select name from Student Where marks
> ALL (Select marks from Student Where
city = 'Karachi');

(4)

Select name from Student Where
student_id NOT IN (Select student_id
from Enrollments);

(5)

Select name from Student s Where
Exists (Select 1 from Enrollments e
Where
e.student_id = s.student_id);

(6)

Select name, (Select MAX(marks) from Student) as max_marks from Students;

(7)

Select course_name from Courses Where credit_hours > (Select AVG(credit_hour) from Courses);

(8)

Select course_name from Courses Where course_id IN (Select course_id from Enrollments Where student_id = (Select student_id from Student Where name = 'Zain'));

(9)

Select name from Student Where student_id IN (Select student_id from Enrollments Where grade IN ('A', 'A+'));

(10)

Select course_name from Courses Where course_id IN (Select course_id from Enrollments Group By course_id Having Count(student_id) > 3);

(11)

Select s1.name from Student s1 Where s1.marks > (Select AVG(s2.marks) from Student s2 where s2.section = s1.section);

(12)

Select course_name from Courses Where credit_hours
= (Select Max(credit_hours) from Courses Where
credit_hours < (Select Max(credit_hours) from Courses));

(13)

Select name from Student Where city IN
(Select city from Student Where student_id
IN (Select student_id from Enrollments
Where course_id = 104));

(14)

Select name from Student Where student_id
IN (Select student_id from Enrollments
Where course_id IN (Select course_id from
Courses Where credit_hours = (Select ~~max~~
Max(credit_hours) from Courses)));

(15)

Select name, (Select COUNT(*) from
Enrollmentse Where e.student_id =
s.student_id) AS enroll-count from
Student s;



Advanced & Correlated

(1)

```
select s1.name, s1.section, s1.marks from Student  
s1 where s1.marks = (select MAX(s2.marks)  
from Student s2 where s2.section = s1.section);
```

(2)

```
select s.name from Student s where exists  
(select 1 from Enrollments e INNER JOIN  
Courses c ON e.course-id = c.course-id WHERE  
e.student-id = s.student-id AND c.credit-hours > 3);
```

(3)

```
select distinct c.course_name from Courses c  
where exists (select 1 from Enrollments e  
INNER JOIN Student s ON e.student-id = s.student  
s.student-id where e.course-id = c.course-id  
AND s.marks > (select AVG(marks) from Student));
```

(4)

```
select s1.name from Student s1 where (select  
COUNT(*) from Enrollments e1 where e1.student-id  
= s1.student-id) < (select AVG(enroll_count)  
from (select COUNT(*) as enroll_count  
from Enrollments Group By student-id) as sub);
```

(5)

```
select s1.name from Student s1 where s1.marks  
> (select MIN(marks) from Student where  
section = 'A') AND s1.marks < (select MAX(marks)  
from Student where section = 'C');
```


MULTI LEVEL NESTED SUBQUERIES

(1)

Select name from Student Where student-id
IN (Select student-id from Enrollments Where
course-id IN (Select course-id from Courses
Where credit-hours = (Select Distinct
credit-hours from Courses ORDER By credit-hours
DESC LIMIT 1 Offset 2));

(2)

Select course-name from Courses Where
course-id IN (Select course-id from Enrollments
Where student-id = (Select student-id
from Student Where city = 'Karachi' AND
marks = (Select MAX(marks) from Student Where
city = 'Karachi'));

(3)

Select name from Student Where marks >
(Select MAX(marks) from Student Where
studentid IN (Select studentid from
Enrollments Where course-id IN (Select
course-id from Courses Where course-name
LIKE '%AI%' OR course-name LIKE '%Machine'));

(4)

Select course-name from Courses Where
course-id NOT IN (Select course-id from Enrollments
Where studentid IN (Select student-id
from Student Where marks < 60));

(5)

Select name from Students where (Select COUNT(*) from Enrollments WHERE e.student-id = s.student-id) = (Select COUNT(*) from Enrollments where student-id = (Select student-id from Student where marks = (Select MAX(marks) from Student)));

Complex EXISTS/NOT EXISTS

(1)

Select s.name from Student s where NOT EXISTS (Select e1.course-id from Enrollments e1 where e1.student-id = 1 AND NOT EXISTS (Select e2.course-id from Enrollments e2 where e2.student-id = s.student-id AND e2.course-id = e1.course-id));

(2)

Select c.course_name from Courses c where NOT EXISTS (Select 1 from Enrollments e where e.course-id = c.course-id AND e.grade = 'A');

(3)

Select s1.name, s1.section from Student s1 where NOT EXISTS (Select 1 from Student s2

Where $s2.section = s1.section$ AND $s2.marks > s1.marks$;

(4)

Select Distinct $s.name$ from Student s WHERE NOT EXISTS (Select 1 from Enrollments e JOIN Courses c ON $e.course_id = c.course_id$ Where $e.student_id = s.student_id$ AND $c.credit_hours \leq$ (Select AVG($credit_hours$) from Courses));

(5)

Select Distinct $s.name$ from Student s Where NOT EXISTS (Select 1 from Enrollments e Where $e.student_id = s.student_id$ AND LENGTH($e.grade$) $\leq$ (Select AVG(LENGTH($e2.grade$)) from Enrollments $e2$ JOIN Student $s2$ ON $e2.student_id = s2.student_id$ Where $s2.section = s.section$));

SUBQUERIES USING ANY

(1)

Select name from Student where marks $>$ ANY (Select marks from Student Where section = 'B');

(2)

Select course_name from Courses Where credit_hours $<$ ANY (Select credit_hours from Courses Where course_name LIKE '%AI%' OR course_name LIKE '%Machine%');

(3)

Select name from Student Where marks >
ANY(Select marks from Student Where city='Lahore');

(4)

Select name from Student Where marks <
ANY(Select marks from Student Where section='c');

(5)

Select name from Student Where marks = ANY
(Select marks from Student Where section='A');

SUBQUERIES USING ALL

(1)

Select name from Students Where marks >
ALL(Select marks from Student where section='A');

(2)

Select name from Student where marks <
ALL(Select marks from Student where section='c');

(3)

Select course_name from Courses where credit_hours
> ALL(Select c.credit_hours from Courses c
JOIN Enrollments e ON c.course_id=e.course_id
Where e.student_id=3);

(4)

Select name from Student Where marks >
All (Select marks from Student Where city='Karachi');

(5)

Select course name from Courses c Where
credit_hours < All (Select credit_hours from
Semester1-Courses); {We don't know Course Semester?}

SUBQUERIES IN FROM CLAUSE

(1)

Select c.course_name, e.enrollment_count from
Courses c JOIN (Select course_id, COUNT(*)
As enrollment_count from Enrollments Group
By course_id) e ON c.course_id = e.course_id;

(2)

Select s.name from Student s JOIN (Select
student_id, COUNT(*) As total from Enrollments
Group By student_id) e ON s.student_id =
e.student_id Where e.total > 2;

(3)

Select t.name, t.marks from (Select name,
marks from Student Order By marks
DESC) t ~~Where~~ LIMIT 3;

(4)

Select d.section, d.avg-marks from (Select section, AVG(marks) as avg-marks from Student Group By section) d;

(5)

Select s.name, counts.student-count from Student s Join Enrollments e ON s.student-id = e.student-id JOIN (select course-id COUNT(student-id) AS student-count from Enrollments Group By course-id) counts ON e.course-id = counts.course-id;

SUBQUERIES IN DELETE

(1)

Delete from Enrollments where student-id IN (Select student-id from Student where marks < (select AVG(marks) from Student));

(2)

Delete from Students where NOT EXISTS (Select 1 from Enrollments e where e.student-id = s.student-id);

(3)

Delete from Course) Where course-id
NOT IN (Select Distinct course-id
from Enrollments);

(4)

Delete from Enrollments Where student-id
IN (Select student-id from Student
Where city = 'Karachi');

(5)

Delete from Student Where marks
< All (Select marks from Student
Where Section = 'A');

11:48pm

12 May